FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA — FIAP

GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

DISCIPLINA: ENGENHARIA DE SOFTWARE

LINCOLN SIMÃO PEREIRA — RM567284

MIGUEL SILVA BEZERRA — RM566763

NICOLAS SAKAUE NISHIMURA — RM567752

GS2026.2 — GLOBAL SOLUTION

NEIGHBORHOOD ACTION DASHBOARD

CitizenTech para Gestão Colaborativa
de Demandas Urbanas

Proposta arquitetural: visão de negócio, arquitetura hexagonal,
visões C4, cenário crítico, decisão arquitetural e estratégia de testes

SÃO PAULO

2026

NEIGHBORHOOD ACTION DASHBOARD

CitizenTech para Gestão Colaborativa de Demandas Urbanas

Documento apresentado como entrega da Global Solution GS2026.2 da disciplina de Engenharia de Software, do curso de Ciência da Computação da FIAP, contendo a concepção arquitetural da plataforma Neighborhood Action Dashboard.

Prof. Dr. Leandro Ramos da Silva

Integrantes da equipe

Nome completo	RM / Matrícula
Lincoln Simão Pereira	RM567284
Miguel Silva Bezerra	RM566763
Nicolas Sakaue Nishimura	RM567752

Repositório do projeto: `https://github.com/lincoln743/GS_SE`

Abordagem arquitetural adotada: Arquitetura Hexagonal (portas e adaptadores)

Visões arquiteturais entregues: Modelo C4 — diagramas de Contexto e de Containers

Versão deste documento: 1.0 — 6 de novembro de 2026

SÃO PAULO

2026

SUMÁRIO

1 INTRODUÇÃO 4
1.1 Apresentação 4
1.2 Escolhas da equipe 4
1.3 Guia de leitura 4
2 ETAPA 1 — VISÃO DO NEGÓCIO 5
2.1 Objetivo da solução 5
2.2 Problema a ser resolvido 5
2.3 Público-alvo 6
2.4 Benefícios esperados 6
2.5 Cenário atual (As-Is) e cenário futuro (To-Be) 7
2.6 Papéis dos usuários 7
2.7 Requisitos funcionais 7
2.8 Requisitos não funcionais 8
3 ETAPA 2 — ARQUITETURA DA SOLUÇÃO 9
3.1 Abordagem adotada 9
3.2 Motivação da escolha 10
3.3 Benefícios 11
3.4 Limitações 11
3.5 Alternativas descartadas 12
3.6 Respostas às perguntas norteadoras 13
4 ETAPA 3 — VISÕES ARQUITETURAIS 13
4.1 Modelo adotado 13
4.2 Diagrama de Contexto (C4 — nível 1) 14
4.3 Diagrama de Containers (C4 — nível 2) 15
4.4 Alinhamento entre as visões e os requisitos 16
5 ETAPA 4 — CENÁRIO CRÍTICO E DIAGRAMA DE SEQUÊNCIA 17
5.1 Cenário escolhido e justificativa 17
5.2 Participantes 17
5.3 Fluxo principal 19
5.4 Fluxos alternativos 20
6 ETAPA 5 — DECISÃO ARQUITETURAL (ADR) 21
7 ETAPA 6 — QUALIDADE E TESTES 22
7.1 Pirâmide de testes 22
7.2 Tipos de teste 23
7.3 Definition of Done 23
7.4 Critério de cobertura 24

8 CONSIDERAÇÕES FINAIS 24
REFERÊNCIAS 26
APÊNDICE A — ADR-002 (REGISTRO ADICIONAL) 27
APÊNDICE B — CHECKLIST FINAL 28

1 INTRODUÇÃO

1.1 Apresentação

A transformação digital ampliou a participação do cidadão na identificação e no acompanhamento dos problemas que afetam o seu bairro. Iniciativas de *CitizenTech* usam a tecnologia para aproximar moradores, organizações e gestores públicos, promovendo transparência, colaboração e decisão apoiada em dados.

Este documento apresenta a concepção arquitetural do **Neighborhood Action Dashboard** (NAD), plataforma digital destinada ao registro, à organização e ao acompanhamento de demandas de melhoria de bairros e comunidades — falhas de iluminação pública, buracos em vias, conservação inadequada de praças e parques, acúmulo de resíduos, problemas de acessibilidade e de sinalização urbana e necessidades de manutenção em equipamentos públicos.

O foco da entrega não é a implementação: é a definição de uma solução consistente sob a perspectiva da Engenharia de Software, com visão de negócio, requisitos, proposta arquitetural, visões, cenário crítico, decisão registrada e estratégia de qualidade.

1.2 Escolhas da equipe

Duas escolhas estruturam o documento e são justificadas nas seções indicadas:

- **abordagem arquitetural: Arquitetura Hexagonal** (portas e adaptadores) — Seção 3;
- **visões arquiteturais: Modelo C4**, com os diagramas de Contexto e de Containers — Seção 4.

Conforme a orientação da atividade, apenas uma abordagem arquitetural e apenas um conjunto de visões foram adotados; as alternativas avaliadas estão registradas com o motivo do descarte.

1.3 Guia de leitura

A Tabela 1 relaciona cada etapa solicitada à seção correspondente deste documento.

Tabela 1 – Correspondência entre as etapas da Global Solution e as seções

Etapa	Seção deste documento	Valor
Etapa 1 — Visão do Negócio	Seção 2, incluindo requisitos funcionais e não funcionais	2,0
Etapa 2 — Arquitetura da Solução	Seção 3, com motivação, benefícios, limitações e alternativas descartadas	2,0
Etapa 3 — Visões Arquiteturais	Seção 4, modelo C4: contexto e containers	2,5
Etapa 4 — Cenário Crítico e Diagrama de Sequência	Seção 5, cenário *Registrar ocorrência*	1,5
Etapa 5 — Decisões Arquiteturais (ADR)	Seção 6, ADR-001	1,5
Etapa 6 — Qualidade e Testes	Seção 7, pirâmide, tipos de teste e Definition of Done	0,5

Fonte: elaborado pelos autores (2026), a partir do enunciado da GS2026.2.

2 ETAPA 1 — VISÃO DO NEGÓCIO

2.1 Objetivo da solução

O Neighborhood Action Dashboard tem por finalidade **centralizar e padronizar o ciclo de vida das demandas de melhoria urbana de um bairro**, desde o registro feito pelo cidadão até o encerramento pelo gestor público, oferecendo a cada parte a informação de que ela precisa: ao cidadão, o andamento do que registrou; ao gestor, a visão consolidada e priorizada do que precisa ser feito; à comunidade, a transparência sobre o que foi resolvido e em quanto tempo.

2.2 Problema a ser resolvido

Hoje as demandas chegam por canais dispersos — telefone, redes sociais, atendimento presencial, aplicativos distintos e mensagens informais — e sem padronização de conteúdo. Três consequências decorrem disso.

A primeira é a **perda de rastreabilidade**: sem protocolo único, nem o cidadão sabe o que aconteceu com o seu pedido, nem a gestão consegue dizer quantas vezes o mesmo problema foi reportado. A segunda é a **priorização frágil**: sem dado estruturado sobre categoria, localização, reincidência e impacto, a fila de atendimento tende a ser ordenada por pressão ou por ordem de chegada, e não por critério. A terceira é a **duplicidade**: o mesmo buraco na mesma rua gera vários registros independentes, que consomem atendimento repetido e distorcem qualquer indicador.

O problema, portanto, não é a falta de canais — é a ausência de um fluxo único, padronizado e transparente que una registro, priorização, execução e prestação de contas.

2.3 Público-alvo

A Tabela 2 identifica quem usa a solução e quem é beneficiado por ela.

Tabela 2 – Público-alvo do Neighborhood Action Dashboard

Grupo	Relação	Interesse na plataforma
Moradores do bairro	Usuário direto	Registrar um problema em menos de dois minutos e saber, sem telefonar para ninguém, em que pé está
Gestores públicos e equipes operacionais	Usuário direto	Receber demandas padronizadas, priorizadas e georreferenciadas, com histórico
Administradores da plataforma	Usuário direto	Manter categorias, usuários e políticas de priorização sem depender de desenvolvimento
Associações de moradores e conselhos locais	Beneficiário	Acompanhar indicadores do bairro e sustentar a cobrança com dado, não com percepção
Comunidade em geral	Beneficiário	Resolução mais ágil, espaço público em melhor estado e transparência sobre o uso do recurso

Fonte: elaborado pelos autores (2026).

2.4 Benefícios esperados

Tabela 3 – Benefícios esperados por grupo de interesse

Para os cidadãos	Para os gestores públicos	Para a comunidade
Registro simples, com foto e localização capturadas do próprio aparelho	Visão consolidada de todas as demandas em um único lugar, sem rastrear canais dispersos	Resolução mais ágil dos problemas do bairro
Protocolo único e acompanhamento transparente, com histórico de situações	Priorização baseada em dados — categoria, reincidência, impacto e tempo de espera	Maior participação nas melhorias locais, porque o registro gera resposta visível
Notificação automática a cada mudança, sem necessidade de cobrar	Indicadores por bairro e por categoria para dimensionar equipes e orçamento	Prestação de contas verificável, com dados abertos sobre o que foi resolvido
Menos retrabalho: ocorrências iguais no mesmo ponto são agrupadas	Menos atendimento duplicado sobre o mesmo problema	Confiança construída sobre evidência, e não sobre promessa

Fonte: elaborado pelos autores (2026).

2.5 Cenário atual (As-Is) e cenário futuro (To-Be)

Tabela 4 – Comparação entre o cenário atual e o cenário futuro

Aspecto	Cenário atual (As-Is)	Cenário futuro (To-Be)
Canal de entrada	Telefone, redes sociais, presencial e aplicativos distintos, sem padrão	Canal único, com formulário padronizado na web e no aplicativo móvel
Identificação	Registro sem protocolo; o cidadão não tem como cobrar	Protocolo gerado no ato, com consulta pública do andamento
Dados da demanda	Texto livre, sem categoria nem localização confiável	Categoria, descrição, foto e coordenada obrigatórias e validadas
Priorização	Por ordem de chegada ou por pressão	Política explícita: categoria, reincidência no mesmo ponto, impacto e tempo de espera
Duplicidade	Cada relato gera um atendimento	Ocorrências próximas da mesma categoria são agrupadas em uma só
Acompanhamento	O cidadão precisa ligar para saber	Notificação automática a cada mudança de situação
Indicadores	Levantamento manual, quando há	Painel por bairro, categoria e tempo médio de resolução, atualizado continuamente
Transparência	Limitada ao que a gestão divulga	Dados abertos exportáveis sobre demandas e prazos

Fonte: elaborado pelos autores (2026).

2.6 Papéis dos usuários

Tabela 5 – Papéis e responsabilidades na plataforma

Papel	Responsabilidades
Cidadão	Registra demandas informando categoria, descrição, localização e fotos; acompanha o andamento pelo protocolo; confirma ou contesta o encerramento
Gestor Público	Analisa as demandas recebidas, ajusta a prioridade sugerida pelo sistema, atualiza a situação com justificativa e prazo, e encerra as solicitações
Administrador	Gerencia usuários e perfis de acesso, mantém as categorias de demanda e configura a política de priorização e os prazos por categoria

Fonte: elaborado pelos autores (2026), conforme os papéis previstos no enunciado da atividade.

2.7 Requisitos funcionais

A Tabela 6 apresenta os requisitos funcionais da solução, acima do mínimo de cinco exigido. A prioridade segue a escala Alta, Média e Baixa.

Tabela 6 – Requisitos funcionais do Neighborhood Action Dashboard

ID	Descrição do requisito	Prioridade
RF01	Registrar ocorrência informando categoria, descrição, localização georreferenciada e fotos, gerando protocolo único de acompanhamento	Alta
RF02	Consultar o andamento de uma ocorrência pelo protocolo, exibindo o histórico completo de situações e as justificativas	Alta
RF03	Classificar e priorizar automaticamente as demandas segundo política configurável de categoria, reincidência no mesmo ponto, impacto e tempo de espera	Alta
RF04	Atualizar a situação da ocorrência — Aberta, Em análise, Em execução, Concluída ou Recusada — com justificativa obrigatória e prazo previsto	Alta
RF05	Exibir painel georreferenciado das demandas, com filtros por bairro, categoria, situação e período	Alta
RF06	Notificar o cidadão por e-mail ou *push* a cada mudança de situação da ocorrência que registrou	Média
RF07	Identificar e agrupar ocorrências duplicadas, isto é, da mesma categoria dentro de um raio configurável e de uma janela de tempo	Média
RF08	Administrar usuários, perfis de acesso, categorias de demanda e prazos por categoria	Média
RF09	Exportar relatórios e dados abertos em CSV e JSON, com indicadores de volume, tempo médio de resolução e taxa de reincidência	Baixa

Fonte: elaborado pelos autores (2026).

2.8 Requisitos não funcionais

A Tabela 7 apresenta os requisitos não funcionais, também acima do mínimo de quatro exigido, cada um associado ao atributo de qualidade correspondente.

Tabela 7 – Requisitos não funcionais e atributos de qualidade

ID	Descrição do requisito	Atributo de qualidade
RNF01	O registro de uma ocorrência deve ser concluído em menos de 2 s no percentil 95, com até 500 usuários simultâneos	Desempenho
RNF02	A plataforma deve manter disponibilidade mensal de 99,5 %, e o aplicativo móvel deve permitir registro sem conexão, sincronizando quando a rede retornar	Disponibilidade e resiliência
RNF03	Dados pessoais do autor da ocorrência não podem ser expostos publicamente; a plataforma deve atender à LGPD quanto a minimização, finalidade e retenção, inclusive nas imagens enviadas	Segurança e privacidade
RNF04	As interfaces devem atender às diretrizes WCAG 2.1 no nível AA, e o registro de uma ocorrência deve ser concluído em no máximo três telas	Usabilidade e acessibilidade
RNF05	O núcleo de domínio não pode depender de framework web, de banco de dados ou de serviço externo, de modo que as regras sejam testáveis sem infraestrutura	Manutenibilidade e testabilidade
RNF06	A plataforma deve suportar a inclusão de novos bairros e o crescimento do volume de ocorrências sem reprojeto, absorvendo picos por meio de processamento assíncrono	Escalabilidade
RNF07	As integrações devem ocorrer por API REST documentada em OpenAPI, de modo que o provedor de mapas, o de identidade e o sistema de protocolo possam ser substituídos	Interoperabilidade

Fonte: elaborado pelos autores (2026).

3 ETAPA 2 — ARQUITETURA DA SOLUÇÃO

3.1 Abordagem adotada

Tabela 8 – Abordagens avaliadas e a escolha da equipe

Escolha	Abordagem	Ideia central
☐	Arquitetura em Camadas	Organização em camadas com responsabilidades bem definidas
✓	**Arquitetura Hexagonal**	**Núcleo de domínio isolado, com comunicação por portas e adaptadores**
☐	Clean Architecture	Dependências apontam para as regras de negócio, no centro da solução
☐	Onion Architecture	Domínio ao centro, envolvido por camadas concêntricas de serviços e infraestrutura

Fonte: elaborado pelos autores (2026), conforme o quadro da GS2026.2.

A Figura 1 representa a abordagem aplicada ao NAD.

Figura 1 – Arquitetura Hexagonal aplicada ao Neighborhood Action Dashboard



Fonte: elaborado pelos autores (2026), conforme Cockburn (2005).

3.2 Motivação da escolha

A característica que mais define este problema é a **multiplicidade de bordas**. A mesma demanda entra por um portal web, por um aplicativo móvel e, no futuro próximo, poderá entrar por um canal de mensagens ou pelo atendimento telefônico transcrito. Ela sai para um provedor de mapas, para um serviço de notificação, para o sistema de protocolo da prefeitura e para um painel de dados abertos. Nenhum desses elementos é estável: provedores municipais mudam, contratos de serviço mudam e canais de atendimento se multiplicam.

A regra de negócio, ao contrário, é estável — o que é uma ocorrência, como ela é priorizada, quais transições de situação são válidas, quando duas ocorrências são a mesma. A Arquitetura Hexagonal existe precisamente para separar o que é estável do que é volátil: o núcleo declara *portas*, interfaces expressas no vocabulário do domínio, e cada tecnologia de borda entra como um *adaptador* que as implementa. Acrescentar o canal de WhatsApp é escrever um novo adaptador primário; trocar o provedor de mapas é escrever um novo adaptador secundário. Em nenhum dos casos a regra de priorização é tocada.

Há ainda uma razão de prova. O requisito RNF05 exige que as regras sejam testáveis sem infraestrutura. Com o núcleo isolado por portas, os testes de unidade da política de priorização

e da máquina de situações rodam em milissegundos, sem subir banco, fila ou servidor web — o que sustenta a base da pirâmide descrita na Seção 7.

3.3 Benefícios

- **Manutenibilidade**: a mudança de uma tecnologia de borda fica confinada a um adaptador; o impacto é local e previsível.
- **Testabilidade**: o domínio é exercitado por testes de unidade puros, com dublês nas portas de saída, sem orquestrar infraestrutura.
- **Evolução por canais**: novos pontos de entrada — canal de mensagens, integração com ouvidoria, importação em lote — não exigem reescrita do núcleo.
- **Substituição de fornecedores**: provedor de mapas, serviço de notificação e sistema de protocolo da prefeitura são implementações intercambiáveis de portas de saída, o que reduz o aprisionamento tecnológico.
- **Clareza de responsabilidade**: a fronteira entre regra de negócio e detalhe técnico fica explícita no código, e não apenas na documentação.

3.4 Limitações

Registrar apenas os benefícios seria apresentar a escolha de forma parcial. A abordagem cobra três preços.

O primeiro é o **custo estrutural inicial**: portas, adaptadores e objetos de transporte entre camadas produzem mais arquivos e mais indireção do que uma organização em camadas simples. Para uma equipe de três integrantes e um prazo curto, isso é trabalho real.

O segundo é a **curva de aprendizado**: a inversão de dependência (“o domínio declara a interface que a infraestrutura implementa”) é contraintuitiva para quem está habituado ao fluxo direto controlador $\rightarrow$ serviço $\rightarrow$ repositório, e exige disciplina em revisão de código para não ser corrompida.

O terceiro é o **risco de excesso**: aplicada sem critério, a abstração gera portas com um único implementador que nunca será substituído — custo sem contrapartida. A equipe adota como regra criar porta apenas quando existir volatilidade real ou necessidade de dublê em teste; os demais casos usam chamada direta.

3.5 Alternativas descartadas

Tabela 9 – Alternativas avaliadas e motivo do descarte

Abordagem	**Por que não foi escolhida**
Arquitetura em Camadas	É a mais simples de implantar e a mais familiar, mas não impede que a regra de negócio dependa do framework e do banco: a camada de serviço costuma importar o ORM diretamente. Como o ponto crítico do NAD é justamente a troca de integrações externas, a abordagem resolveria o problema mais fácil e deixaria o mais difícil
Clean Architecture	Atenderia ao problema: o efeito prático sobre a dependência é equivalente ao da Hexagonal. Foi descartada por acrescentar camadas prescritivas — entidades, casos de uso, adaptadores de interface e frameworks — cujo ganho, no tamanho deste sistema, não compensa a cerimônia adicional. A Hexagonal entrega a mesma inversão com menos estrutura
Onion Architecture	Igualmente próxima em princípio, com o domínio ao centro e camadas concêntricas. Foi descartada pela mesma razão da Clean, somada à menor familiaridade da equipe com o seu vocabulário, o que aumentaria o custo de comunicação sem benefício arquitetural correspondente

Fonte: elaborado pelos autores (2026).

Registre-se que Clean e Onion não foram descartadas por serem inadequadas — elas não são. Foram descartadas por custo de estrutura diante de um ganho equivalente, e essa é a razão honesta.

3.6 Respostas às perguntas norteadoras

Tabela 10 – Perguntas norteadoras da Etapa 2 e as respostas da equipe

Pergunta	Resposta
Quais características de qualidade são mais críticas para o NAD?	Manutenibilidade e testabilidade, pela volatilidade das integrações municipais; em seguida, disponibilidade e privacidade, porque o sistema recebe dado pessoal e imagem de via pública
Como a abordagem separa as regras de negócio da interface, do banco e das integrações?	O núcleo depende apenas das portas que ele mesmo declara (`OcorrenciaRepository`, `EventoPublisher`, `GeocodificacaoGateway`, `ProtocoloGateway`); controladores, repositórios e clientes externos são implementações dessas interfaces, resolvidas por injeção de dependência
Que complexidade adicional a abordagem traz, considerando o tamanho da equipe e o prazo?	Mais arquivos, mais indireção e necessidade de disciplina em revisão. Mitigação: criar porta apenas onde houver volatilidade ou necessidade de dublê, e manter um documento curto de convenções no repositório
Por que cada uma das demais abordagens foi descartada?	Camadas, por não proteger o domínio das integrações; Clean e Onion, por custo de estrutura superior ao ganho neste porte de sistema — conforme a Tabela 9
Como a escolha se conecta às visões arquiteturais apresentadas?	O núcleo hexagonal vive dentro do container *API de Demandas* do diagrama de containers; os adaptadores secundários são exatamente as setas que saem desse container para o banco, a fila e os sistemas externos (Seção 4)

Fonte: elaborado pelos autores (2026).

4 ETAPA 3 — VISÕES ARQUITETURAIS

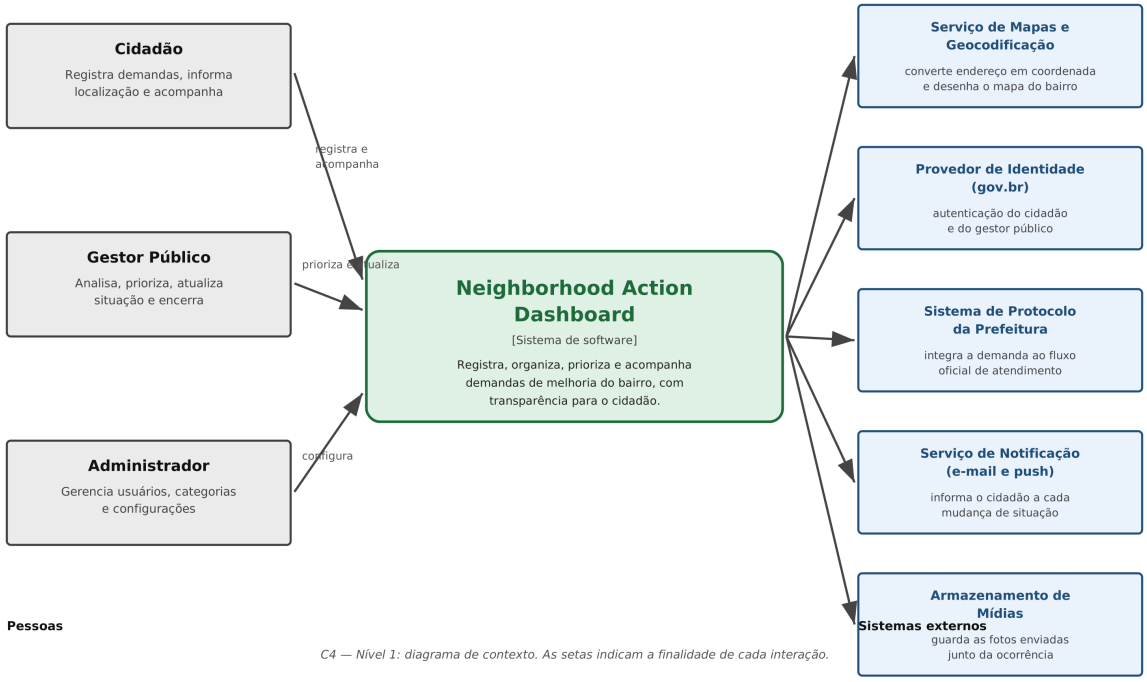
4.1 Modelo adotado

A equipe adota o **Modelo C4**, proposto por Brown, entregando os diagramas de **Contexto** (nível 1) e de **Containers** (nível 2), conforme previsto na Opção A da atividade. O Modelo 4+1 de Kruchten não foi utilizado.

A escolha do C4 decorre de dois fatores. O primeiro é a aderência ao que precisa ser comunicado nesta etapa: o contexto responde *quem usa o sistema e com que sistemas ele conversa*, e o de containers responde *em que unidades implantáveis a solução se divide e como elas se comunicam* — exatamente as duas perguntas que a arquitetura hexagonal escolhida precisa responder para ser verificável. O segundo é a economia de notação: o C4 se apoia em poucos elementos e torna o diagrama legível por quem não é da equipe, o que importa em um sistema cujo interlocutor inclui gestores públicos.

4.2 Diagrama de Contexto (C4 — nível 1)

Figura 2 – C4 nível 1 — diagrama de contexto do Neighborhood Action Dashboard



Fonte: elaborado pelos autores (2026), conforme Brown (2024).

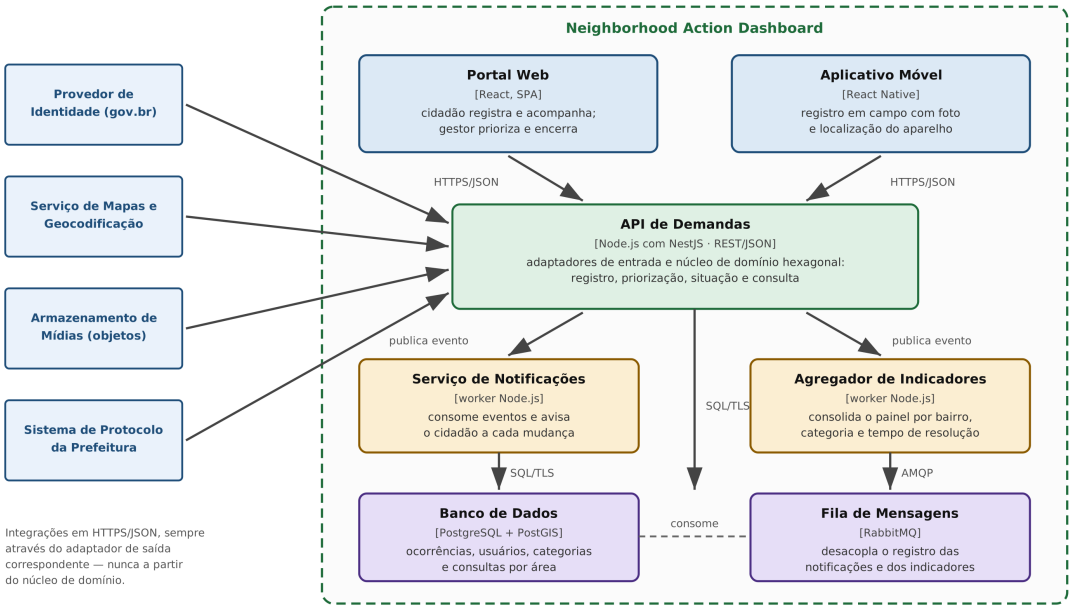
Tabela 11 – Elementos do diagrama de contexto

Elemento	Tipo	Finalidade da interação
Cidadão	Pessoa	Registra demandas, informa a localização e acompanha o andamento pelo protocolo
Gestor Público	Pessoa	Analisa, prioriza, atualiza a situação e encerra solicitações
Administrador	Pessoa	Gerencia usuários, categorias e configurações da plataforma
Provedor de Identidade (gov.br)	Sistema externo	Autentica cidadãos e gestores, dispensando a plataforma de gerir senhas
Serviço de Mapas e Geocodificação	Sistema externo	Converte endereço em coordenada e fornece a base cartográfica do painel
Sistema de Protocolo da Prefeitura	Sistema externo	Recebe a demanda e a integra ao fluxo oficial de atendimento do município
Serviço de Notificação	Sistema externo	Entrega e-mail e *push* ao cidadão a cada mudança de situação
Armazenamento de Mídias	Sistema externo	Guarda as fotos anexadas à ocorrência, fora do banco relacional

Fonte: elaborado pelos autores (2026).

4.3 Diagrama de Containers (C4 — nível 2)

Figura 3 – C4 nível 2 — diagrama de containers do Neighborhood Action Dashboard



C4 — Nível 2: diagrama de containers. Cada caixa é uma unidade implantável; os rótulos indicam o protocolo de comunicação.
O núcleo de domínio hexagonal vive dentro da API de Demandas; os demais containers o alcançam apenas por portas.

Fonte: elaborado pelos autores (2026), conforme Brown (2024).

Tabela 12 – Containers, tecnologias e responsabilidades

Container	Tecnologia	Responsabilidade e comunicação
Portal Web	React (SPA)	Interface do cidadão e do gestor; consome a API de Demandas por HTTPS/JSON
Aplicativo Móvel	React Native	Registro em campo com foto e coordenada do aparelho; fila local para operação sem rede; mesmo contrato REST do portal
API de Demandas	Node.js com NestJS	Hospeda os adaptadores de entrada e o núcleo de domínio hexagonal: registro, priorização, transição de situação e consulta. Acessa o banco por SQL/TLS e publica eventos na fila
Serviço de Notificações	Worker Node.js	Consome o evento `OcorrenciaRegistrada` e os eventos de mudança de situação e aciona o serviço externo de notificação
Agregador de Indicadores	Worker Node.js	Consolida o painel por bairro, categoria e tempo de resolução a partir dos eventos, sem onerar a API
Banco de Dados	PostgreSQL com PostGIS	Persiste ocorrências, usuários e categorias; responde às consultas por área, que são a base do painel georreferenciado
Fila de Mensagens	RabbitMQ	Desacopla o registro das notificações e dos indicadores, absorvendo picos de demanda

Fonte: elaborado pelos autores (2026).

4.4 Alinhamento entre as visões e os requisitos

A Tabela 13 liga cada container aos requisitos que ele atende, para tornar verificável a afirmação de que a arquitetura responde ao que foi especificado na Etapa 1.

Tabela 13 – Rastreabilidade entre containers e requisitos

Container	Requisitos atendidos	Observação
Portal Web	RF01, RF02, RF04, RF05, RF08; RNF04	Concentra a exigência de acessibilidade WCAG 2.1 AA
Aplicativo Móvel	RF01, RF02; RNF02	A fila local sustenta o registro sem conexão
API de Demandas	RF01 a RF08; RNF01, RNF05, RNF07	Abriga o núcleo de domínio; é onde a política de priorização e o agrupamento de duplicadas vivem
Serviço de Notificações	RF06	Assíncrono, para não atrasar a resposta do registro
Agregador de Indicadores	RF05, RF09	Pré-consolida os indicadores do painel e da exportação
Banco de Dados	RF02, RF05, RF07; RNF03	PostGIS viabiliza o raio de agrupamento e o filtro por bairro
Fila de Mensagens	RF06, RF07; RNF06	Absorve picos e isola falhas das integrações externas

Fonte: elaborado pelos autores (2026).

5 ETAPA 4 — CENÁRIO CRÍTICO E DIAGRAMA DE SEQUÊNCIA

5.1 Cenário escolhido e justificativa

O cenário crítico modelado é **Registrar ocorrência**. A escolha se justifica por três razões: é o caso de uso de maior volume, pois todo o restante do sistema depende de ele existir; é o que atravessa o maior número de participantes, da interface ao domínio, ao banco e à mensageria; e é o único em que a falha é diretamente visível para o cidadão — se o registro se perde, a plataforma perde o seu propósito e a confiança que pretende construir.

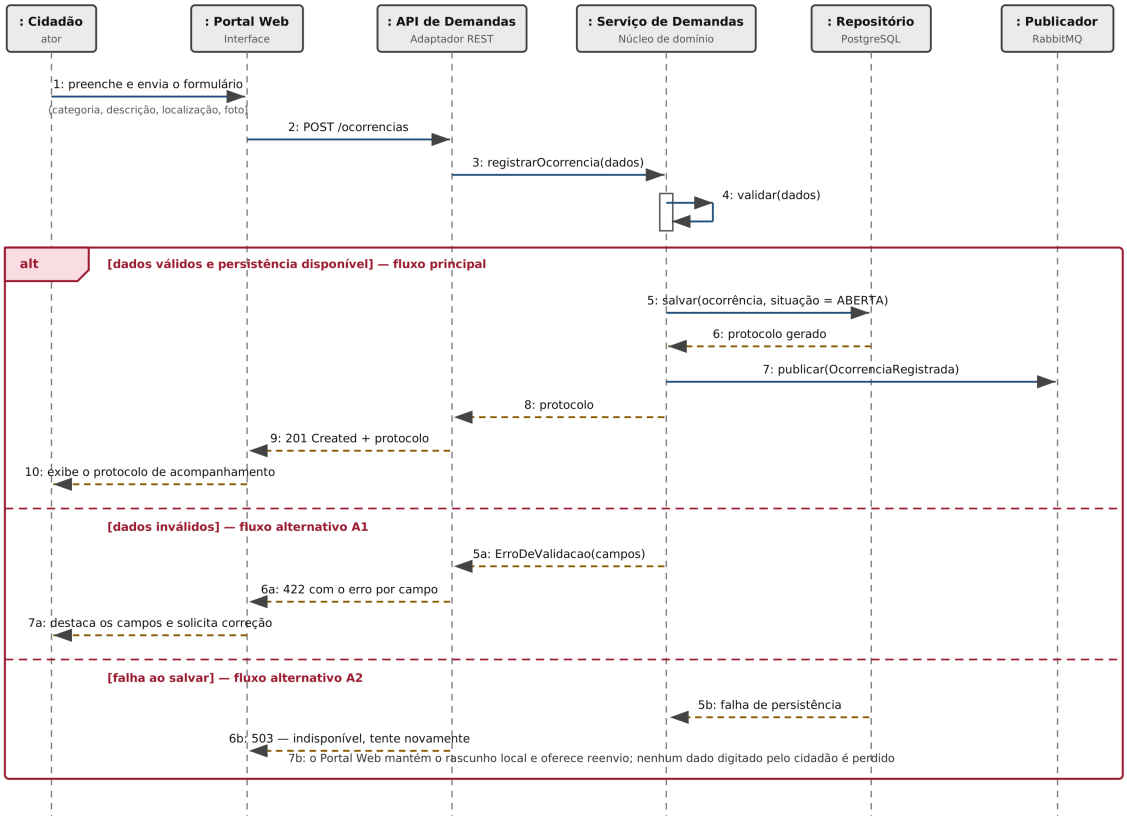
5.2 Participantes

Tabela 14 – Participantes do cenário crítico

Participante	**Papel no cenário**
Cidadão	Ator que inicia o fluxo preenchendo o formulário
Portal Web	Interface que coleta os dados e trata o retorno
API de Demandas	Adaptador primário REST; recebe a requisição e a traduz para a porta de entrada do domínio
Serviço de Demandas	Núcleo de domínio; valida os dados e aplica as regras de registro
Repositório	Adaptador secundário de persistência sobre PostgreSQL
Publicador	Adaptador secundário de mensageria sobre RabbitMQ

Fonte: elaborado pelos autores (2026).

Figura 4 – Diagrama de sequência do cenário Registrar ocorrência



Fonte: elaborado pelos autores (2026), conforme Object Management Group (2017).

5.3 Fluxo principal

Tabela 15 – Mensagens do fluxo principal — Registrar ocorrência

#	De	Para	Mensagem
1	Cidadão	Portal Web	Preenche o formulário (categoria, descrição, localização, foto) e envia
2	Portal Web	API de Demandas	`POST /ocorrencias` com os dados da ocorrência
3	API de Demandas	Serviço de Demandas	`registrarOcorrencia(dados)` pela porta de entrada
4	Serviço de Demandas	Serviço de Demandas	`validar(dados)`: campos obrigatórios, categoria existente e coordenada dentro da área atendida
5	Serviço de Demandas	Repositório	`salvar(ocorrência, situação = ABERTA)`
6	Repositório	Serviço de Demandas	Retorna o protocolo gerado
7	Serviço de Demandas	Publicador	`publicar(OcorrenciaRegistrada)` para notificação e indicadores
8	Serviço de Demandas	API de Demandas	Retorna o protocolo
9	API de Demandas	Portal Web	`201 Created` com o protocolo
10	Portal Web	Cidadão	Exibe o protocolo de acompanhamento

Fonte: elaborado pelos autores (2026).

5.4 Fluxos alternativos

Tabela 16 – Fluxos alternativos do cenário

#	De	Para	Mensagem
A1	Serviço de Demandas	Portal Web (via API)	[dados inválidos] Retorna `422` com o erro por campo; a interface destaca os campos e solicita correção, preservando o que já foi digitado
A2	Repositório	Portal Web (via API)	[falha ao salvar] Retorna `503`; a interface informa a indisponibilidade, mantém o rascunho local e oferece novo envio, de modo que nenhum dado do cidadão é perdido
A3	Serviço de Demandas	Portal Web (via API)	[ocorrência duplicada] Quando já existe demanda aberta da mesma categoria no raio configurado, o sistema vincula o novo relato à ocorrência existente e devolve o protocolo dela, informando o cidadão de que o problema já está em acompanhamento

Fonte: elaborado pelos autores (2026).

O fluxo A3 merece nota, por ser decisão de projeto e não apenas tratamento de erro: ele implementa o requisito RF07 no momento do registro, e não em uma rotina posterior de limpeza. Evitar a duplicata na entrada custa uma consulta geoespacial; removê-la depois custa atendimento repetido e indicador distorcido.

6 ETAPA 5 — DECISÃO ARQUITETURAL (ADR)

Tabela 17 – ADR-001 — Adoção da Arquitetura Hexagonal no núcleo de demandas

Status / Data	**Aceita** (☐ Proposta ✓ Aceita ☐ Substituída) Data: 06/11/2026
Contexto	O Neighborhood Action Dashboard recebe demandas por múltiplos canais — portal web e aplicativo móvel hoje, canal de mensagens e importação da ouvidoria em estudo — e depende de quatro integrações externas sobre as quais a equipe não tem governança: provedor de identidade, serviço de mapas, sistema de protocolo da prefeitura e serviço de notificação. Contratos municipais são revistos periodicamente, e a substituição de qualquer um desses fornecedores é provável dentro do horizonte do projeto. Ao mesmo tempo, as regras de negócio — política de priorização, máquina de situações e critério de duplicidade — são estáveis e precisam ser testadas sem infraestrutura, conforme o RNF05. A equipe tem três integrantes e prazo curto
Decisão	Decidimos adotar a Arquitetura Hexagonal (portas e adaptadores) no container *API de Demandas*. O núcleo de domínio declara as portas de entrada (`RegistrarOcorrencia`, `AtualizarSituacao`, `PriorizarDemanda`, `ConsultarPainel`) e as portas de saída (`OcorrenciaRepository`, `EventoPublisher`, `GeocodificacaoGateway`, `ProtocoloGateway`); framework web, persistência, mensageria e clientes externos são adaptadores que implementam essas interfaces. Nenhuma classe do núcleo importa biblioteca de infraestrutura
Alternativas consideradas	**Arquitetura em Camadas** — descartada por não impedir que a regra de negócio dependa do ORM e do framework, deixando sem solução justamente o ponto crítico do sistema. **Clean Architecture** — adequada e equivalente em efeito, descartada pelo número maior de camadas prescritivas, cuja cerimônia não se paga neste porte. **Onion Architecture** — próxima em princípio, descartada pela mesma razão, somada à menor familiaridade da equipe com o seu vocabulário
Consequências positivas	Troca de fornecedor externo passa a ser a escrita de um adaptador, sem alteração de regra de negócio; testes de unidade do domínio rodam sem banco, fila ou servidor, sustentando a base da pirâmide de testes; novos canais de entrada não exigem reescrita do núcleo; a fronteira entre negócio e tecnologia fica explícita no código e é verificável em revisão
Consequências negativas	Mais arquivos e mais indireção do que uma organização em camadas, com custo real para uma equipe de três integrantes; curva de aprendizado da inversão de dependência, que exige disciplina em revisão de código para não ser corrompida; risco de portas com implementador único, sem ganho — mitigado pela regra de criar porta apenas onde houver volatilidade ou necessidade de dublê em teste

Fonte: elaborado pelos autores (2026), conforme o modelo de Nygard (2011).

O Apêndice A traz um segundo registro, não exigido pela atividade, sobre a adoção de mensageria assíncrona — decisão que aparece no diagrama de containers e que, por consistência, também convém documentar.

7 ETAPA 6 — QUALIDADE E TESTES

7.1 Pirâmide de testes

A Figura 5 apresenta a estratégia de testes do NAD organizada pela pirâmide. A regra de alocação é direta: o que é regra de negócio pura fica na base; o que depende de recurso externo real fica no meio; e só sobe ao topo o que precisa atravessar o sistema inteiro pela perspectiva do usuário.

Figura 5 – Pirâmide de testes do Neighborhood Action Dashboard



Fonte: elaborado pelos autores (2026), conforme Cohn (2009).

7.2 Tipos de teste

Tabela 18 – Níveis de teste e exemplos no Neighborhood Action Dashboard

Nível	**Objetivo**	**Exemplo no NAD**
Testes Unitários	Verificar, de forma isolada, regras de negócio e funções individuais	Política de priorização de uma demanda: categoria crítica com reincidência no mesmo ponto deve resultar em prioridade alta. Validação dos campos obrigatórios da ocorrência. Transição proibida de *Aberta* para *Concluída* sem passar por *Em execução*
Testes de Integração	Verificar a comunicação entre componentes, serviços e persistência	A API de ocorrências gravando e consultando no PostgreSQL, com a consulta geoespacial que encontra demandas no raio de agrupamento; publicação e consumo do evento `OcorrenciaRegistrada` na fila
Testes End-to-End	Verificar fluxos completos sob a perspectiva do usuário	O cidadão registra uma ocorrência e recebe o protocolo; o gestor prioriza, atualiza a situação e encerra; o cidadão consulta o andamento e encontra o histórico completo

Fonte: elaborado pelos autores (2026).

7.3 Definition of Done

A Tabela 19 define os critérios mínimos para considerar uma funcionalidade pronta. A coluna *Cumprido* é preenchida a cada funcionalidade entregue; a coluna *Evidência* registra o que comprova o atendimento — sem evidência, o critério não é considerado cumprido.

Tabela 19 – Definition of Done do Neighborhood Action Dashboard

Critério mínimo	**Cumprido**	**Evidência exigida**
Requisito implementado e critérios de aceite validados	☐	Critérios de aceite da história marcados e demonstrados
Testes unitários escritos e aprovados	☐	Execução verde no pipeline e cobertura do pacote de domínio sem queda
Testes de integração aprovados	☐	Execução verde com banco e fila em contêineres efêmeros
Fluxo principal validado em teste End-to-End	☐	Execução da jornada correspondente registrada
Código revisado por outro integrante da equipe	☐	*Pull request* aprovado por um par
Decisão arquitetural registrada em ADR, quando aplicável	☐	ADR criado ou atualizado no repositório
Sem defeitos críticos ou bloqueantes em aberto	☐	Quadro de defeitos sem item de severidade 1 ou 2 associado
Documentação da funcionalidade atualizada	☐	Contrato OpenAPI e anotações de uso revisados

Fonte: elaborado pelos autores (2026), a partir do modelo proposto na GS2026.2.

7.4 Critério de cobertura

A meta de cobertura é diferenciada por pacote: 80 % de instruções e 70 % de ramos no núcleo de domínio, 60 % nos adaptadores, e nenhuma meta para código gerado e objetos de transporte. A regra que acompanha a meta importa mais do que o número: a cobertura do domínio não pode cair de uma entrega para a seguinte.

Vale a ressalva, porém, de que **cobertura mede execução, não detecção**: linhas percorridas por asserções fracas produzem a aparência de qualidade. Por isso a Definition of Done exige critérios de aceite validados, e não apenas testes verdes.

8 CONSIDERAÇÕES FINAIS

A proposta aqui apresentada parte de um diagnóstico simples: o problema do Neighborhood Action Dashboard não é a falta de canais para o cidadão reclamar — canais existem e se multiplicam. O problema é a ausência de um fluxo único, padronizado e transparente, que transforme relato disperso em demanda rastreável, priorizada por critério e verificável por quem reclamou.

A arquitetura escolhida responde ao traço mais marcante desse contexto: a instabilidade das bordas. Canais de entrada se multiplicam e fornecedores municipais mudam, enquanto a regra de negócio permanece. A Arquitetura Hexagonal foi adotada exatamente para que a volatilidade

fique confinada aos adaptadores — e o documento registra também o que essa escolha custa, porque uma decisão apresentada só pelos benefícios não é uma decisão documentada, é uma defesa.

As visões em C4 tornam essa escolha verificável: o diagrama de contexto mostra que quatro dos oito elementos externos são serviços substituíveis, e o de containers mostra que todos eles são alcançados a partir de um único container, pela via dos adaptadores. O cenário de registro de ocorrência, por sua vez, mostra a arquitetura em funcionamento, inclusive naquilo que costuma ficar de fora de uma proposta: o que acontece quando o dado é inválido, quando a persistência falha e quando o problema já tinha sido reportado por outra pessoa.

Registre-se, por fim, o limite desta entrega. Trata-se de concepção arquitetural, não de implementação: não há código, não há medição de desempenho e os números de requisito não funcional são metas de projeto, não resultados. A validação desses valores depende de protótipo executável e de uso real, que são o passo seguinte natural deste trabalho.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro, 2018.

BRASIL. **Lei n.º 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília: Presidência da República, 2018.

BROWN, Simon. **The C4 model for visualising software architecture**. Disponível em: https://c4model.com. Acesso em: 6 nov. 2026.

COCKBURN, Alistair. **Hexagonal architecture**: ports and adapters. 2005. Disponível em: https://alistair.cockburn.us/hexagonal-architecture. Acesso em: 6 nov. 2026.

COHN, Mike. **Succeeding with Agile**: software development using Scrum. Boston: Addison-Wesley, 2009.

EVANS, Eric. **Domain-Driven Design**: tackling complexity in the heart of software. Boston: Addison-Wesley, 2003.

FOWLER, Martin. **Refactoring**: improving the design of existing code. 2. ed. Boston: Addison-Wesley, 2019.

KRUCHTEN, Philippe. Architectural blueprints: the 4+1 view model of software architecture. **IEEE Software**, v. 12, n. 6, p. 42-50, 1995.

MARTIN, Robert C. **Arquitetura limpa**: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

NYGARD, Michael. **Documenting architecture decisions**. 2011. Cognitect Blog. Disponível em: https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions. Acesso em: 6 nov. 2026.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language (OMG UML)**, version 2.5.1. 2017. Especificação técnica formal/2017-12-05.

PALERMO, Jeffrey. **The Onion Architecture**. 2008. Disponível em: https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1. Acesso em: 6 nov. 2026.

SILVA, Leandro Ramos da. **GS2026.2 — Global Solution**: Neighborhood Action Dashboard — CitizenTech para gestão colaborativa de demandas urbanas. São Paulo: FIAP, 2026. Material de apoio da disciplina de Engenharia de Software.

WORLD WIDE WEB CONSORTIUM. **Web Content Accessibility Guidelines (WCAG) 2.1**. W3C Recommendation, 2018. Disponível em: https://www.w3.org/TR/WCAG21. Acesso em: 6 nov. 2026.

APÊNDICE A — ADR-002 (REGISTRO ADICIONAL)

O registro a seguir não é exigido pela atividade, que pede um ADR. Ele é incluído porque a decisão aparece no diagrama de containers e porque a prática adotada pela equipe é documentar toda decisão com consequência estrutural.

Tabela 20 – ADR-002 — Processamento assíncrono de notificações e indicadores

Status / Data	**Aceita** Data: 06/11/2026
Contexto	O registro de uma ocorrência precisa responder em menos de 2 s (RNF01), mas dispara duas atividades que dependem de terceiros ou de agregação: notificar o cidadão e atualizar os indicadores do painel. Executá-las dentro da requisição significa que a lentidão ou a queda do serviço de notificação se transforma em falha no registro — justamente a operação que não pode falhar
Decisão	Publicar o evento `OcorrenciaRegistrada` em fila de mensagens e tratar notificação e indicadores em *workers* independentes, de modo que a resposta ao cidadão dependa apenas da persistência
Alternativas consideradas	Chamada síncrona dentro da requisição — descartada por acoplar a disponibilidade do registro à de terceiros. Rotina periódica em lote — descartada porque atrasaria a notificação em minutos e removeria o efeito de transparência imediata que justifica a plataforma
Consequências positivas	O registro continua funcionando com o serviço de notificação indisponível; picos de demanda são absorvidos pela fila; novos consumidores do evento podem ser acrescentados sem alterar o fluxo de registro
Consequências negativas	Acrescenta um componente de infraestrutura a operar e monitorar; introduz consistência eventual entre o registro e o painel de indicadores; exige consumidores idempotentes, porque a entrega da mensagem pode se repetir

Fonte: elaborado pelos autores (2026), conforme o modelo de Nygard (2011).

APÊNDICE B — CHECKLIST FINAL

Tabela 21 – Conferência dos itens obrigatórios do documento

Item	Exigência	Onde está neste documento
✓	Capa	Página de rosto do documento
✓	Integrantes	Folha de rosto, com nome completo e RM
✓	Documento de Visão	Seção 2 — objetivo, problema, público-alvo, benefícios, As-Is/To-Be, papéis e requisitos
✓	Arquitetura da Solução	Seção 3 — Arquitetura Hexagonal, com motivação, benefícios, limitações e alternativas descartadas
✓	C4 ou 4+1	Seção 4 — Modelo C4, diagramas de contexto e de containers
✓	Diagrama de Sequência	Seção 5 — cenário *Registrar ocorrência*, com fluxo principal e três fluxos alternativos
✓	ADR	Seção 6 — ADR-001; Apêndice A traz o ADR-002, adicional
✓	Estratégia de Testes	Seção 7 — pirâmide, três níveis de teste e Definition of Done
✓	Referências	Lista de referências conforme a ABNT NBR 6023

Fonte: elaborado pelos autores (2026), conforme o checklist final da GS2026.2.